INTRODUCING

SlideKit

Coordinate-based slide layout for the AI era

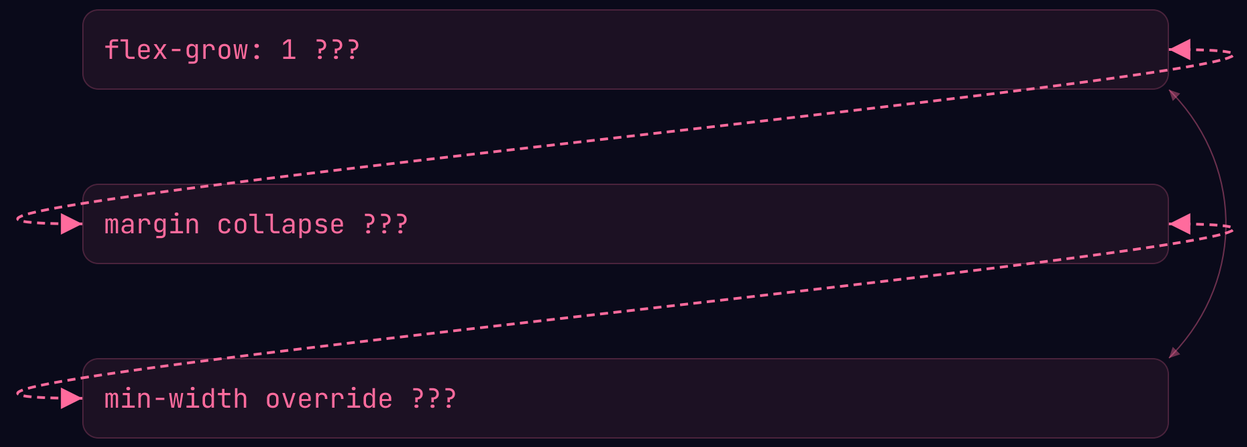
Deterministic · Measurable · Validated

THE PROBLEM

CSS Wasn’t Built for Slides

CSS layout was designed for reflowing documents that adapt to variable screen sizes. Slides are the opposite — fixed canvases where every element has a deliberate position.

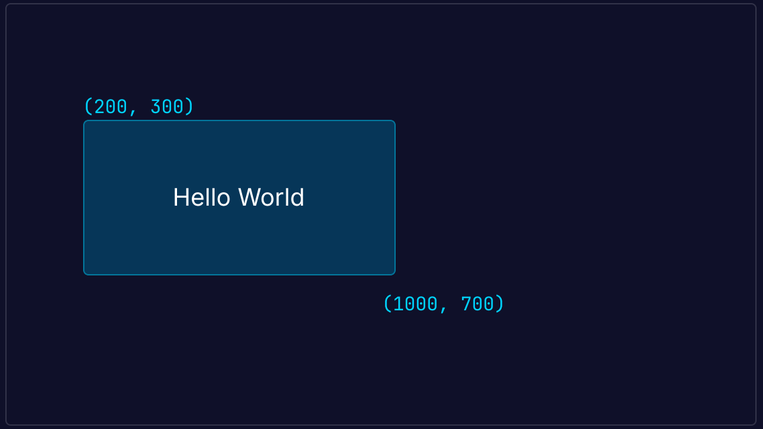
When you use auto-layout for slides, the **browser decides** where things go. When it decides wrong, you debug cascading interactions and emergent reflow.



THE SOLUTION

Just Say Where Things Go

```
el('Hello World', {
  x: 200, y: 300,
  w: 800, h: 400
})
```



“You say where. It goes there. No surprises.”

DESIGNED FOR AI

LLMs Think in Coordinates

❌ CSS Approach

```
display: flex;
gap: 1rem;
flex-wrap: wrap;
min-width: 200px;
```

→ Where does it end up? 🤷

✅ SlideKit Approach

```
x: 200, y: 300,
w: 800, h: 400
```

→ Exactly at (200, 300). Always.

"AI agents can reason about geometry. They cannot simulate browser layout engines."

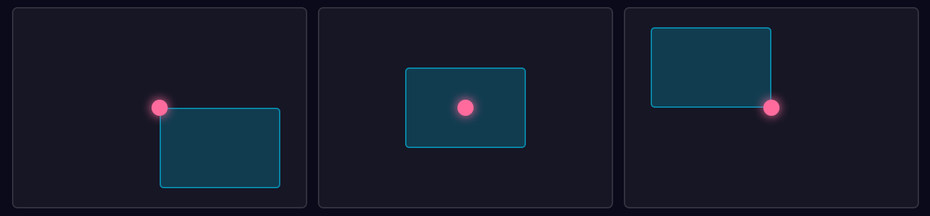
POSITIONING

The Anchor System

Nine points of control for precise element placement

Each anchor determines which point of the element sits at (x, y)

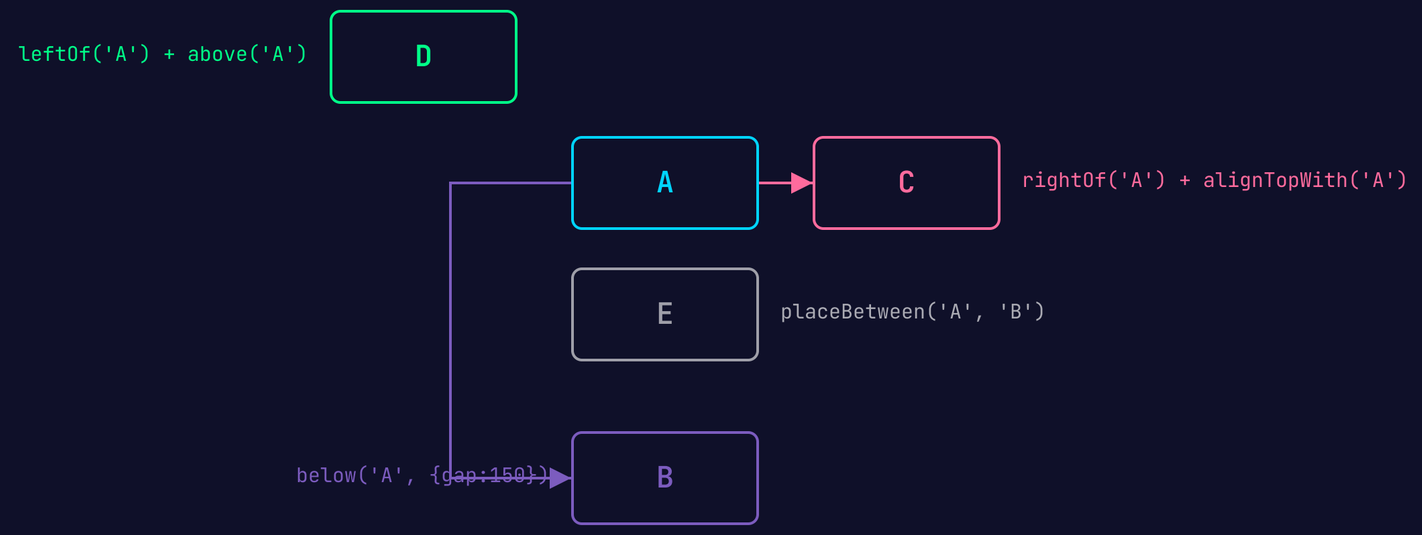
Same (x, y), different anchor:



`anchor: 'tl'` `anchor: 'cc'` `anchor: 'br'`

RELATIVE POSITIONING

Elements That Know About Each Other



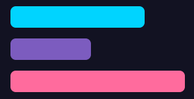
Position a label dead-center inside any rect

STACK LAYOUTS

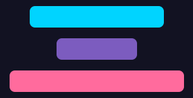
Vertical & Horizontal Flow

vstack

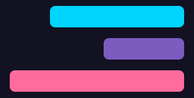
left



center



right



stretch

hstack

top

middle

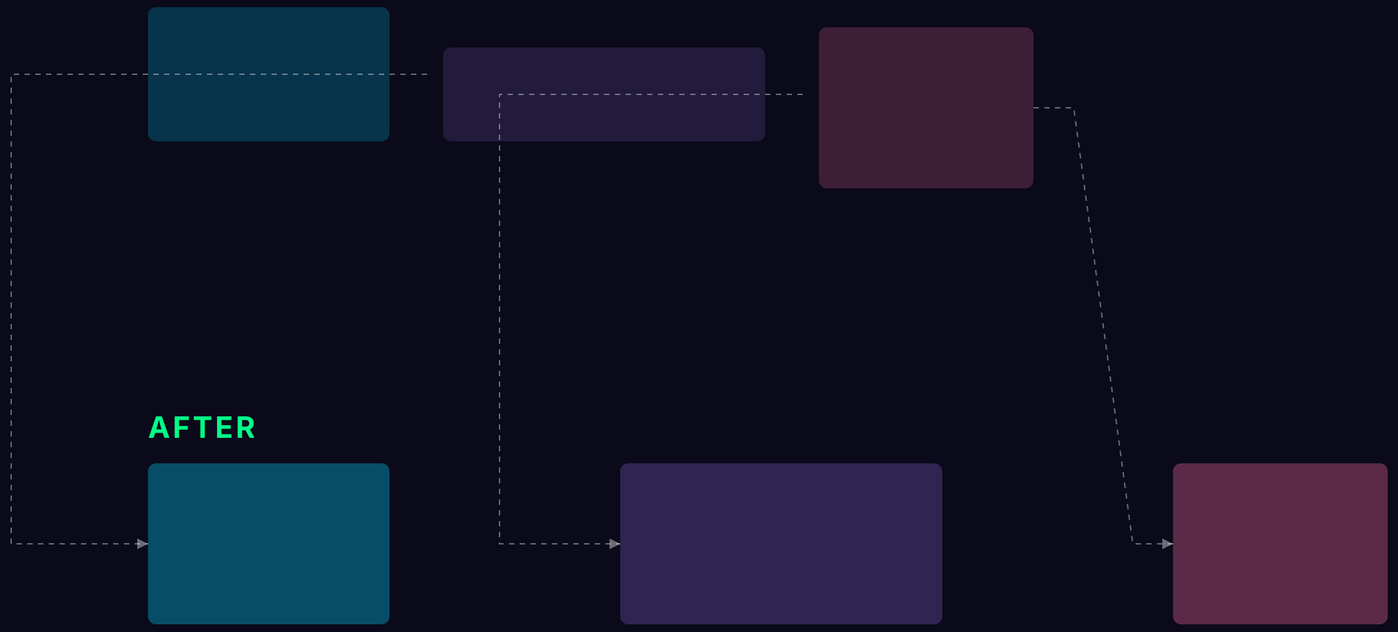
bottom

stretch

TRANSFORMS

PowerPoint-Style Precision

BEFORE



AFTER

```
transforms: [
  alignTop([...]),
  distributeH([...]),
  matchHeight([...])
]
```

FITTORECT()

STRUCTURED CONTENT

Panels & Card Grids

Measurement

Every element has a resolved bounding box. Query positions, check overlaps, validate alignment.

Validation

Built-in warnings for overflow, overlap, tiny text, and missing fonts — before you render.

Layering

Explicit layer ordering: bg → default → fg → overlay. No z-index guessing games.

Speaker Notes

Attach notes to any slide. Rendered in presenter view, excluded from PDF export.

Debug Overlay

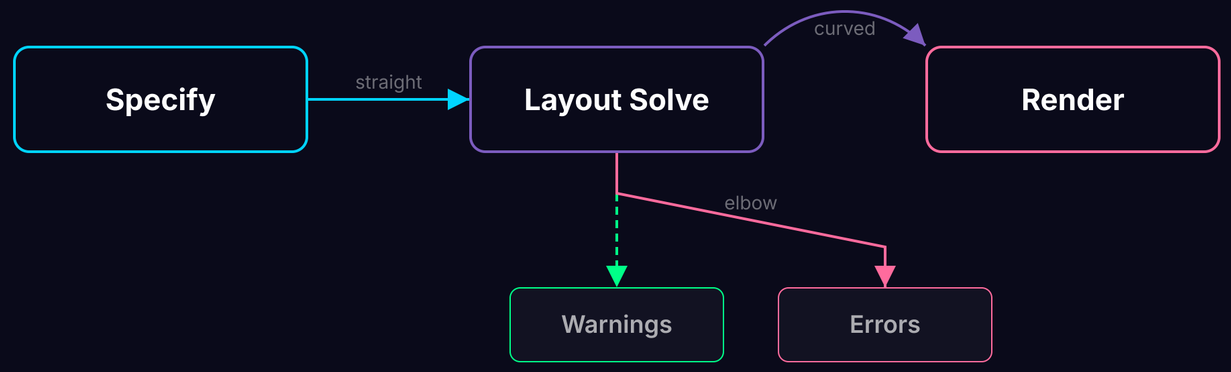
Toggle bounding-box outlines, safe-zone markers, and element IDs for rapid debugging.

Theming

Define design tokens once. Reference them everywhere. Change the palette in one place.

CONNECTORS

Draw Relationships

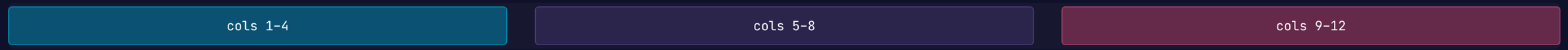


Three connector types: straight · curved · elbow — with arrow, dash & label options

UTILITIES

Grid, Snap & Repeat

grid()



grid({ cols: 12, gutter: 30 }) → g.col(1), g.spanW(1, 4)

snap()

157 → 160 snap(v, 10) 243 → 240

snap(157, 10) → 160 | snap(243, 10) → 240

repeat()

repeat(dot, { count: 12, cols: 4, gapX: 28, gapY: 28 })

COMPOUND ELEMENTS

Figures

Structured visual containers with padding, radius & captions

containerFill

containerRadius + Padding

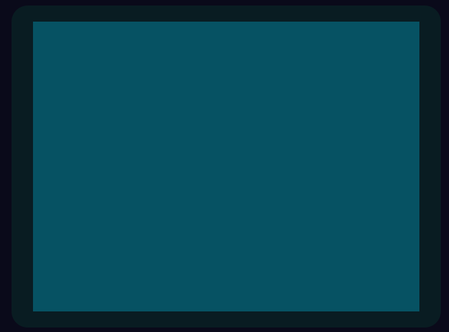
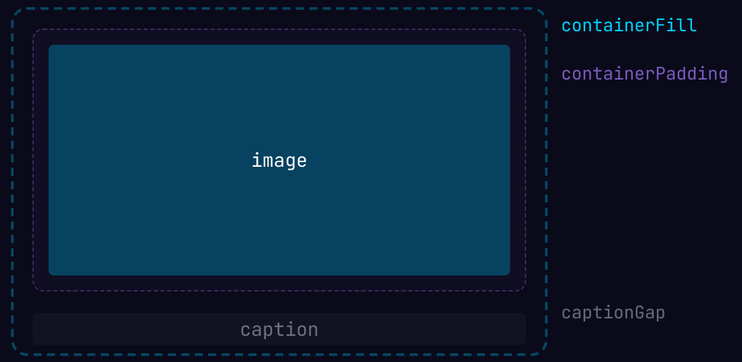


Fig 3 — with caption

caption + captionGap



VISUAL DESIGN

You Own the Pixels

Gradients

Shadow Presets

sm

md

lg

xl

glow

Glass-morphism

Glass panel overlay

Borders & Overflow

solid border

dashed border

overflow: clip

QUALITY

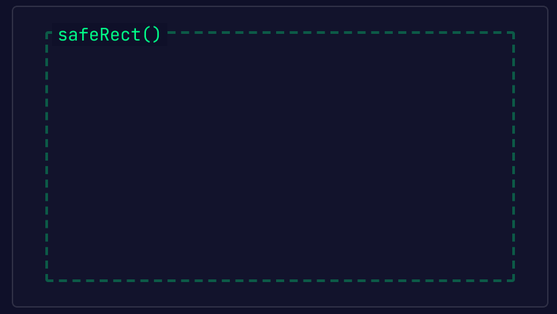
Catch Problems Early

measure()

```
const size = await measure(
  '<h1>Hello World</h1>',
  { w: 800, fontSize: 56 }
);
// → { w: 482, h: 64 }
```

Validation Output

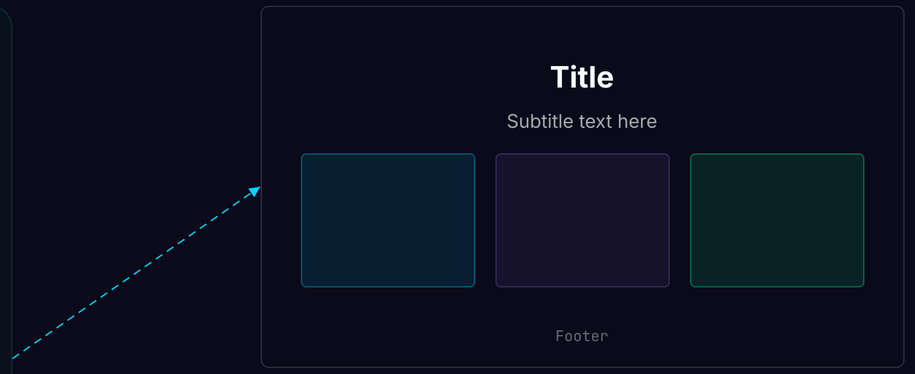
```
⚠ Element "card-3" extends beyond safe zone (x: 1820 + w: 300 > 1800)

✗ Element "title" missing required "w" property

✓ 14 elements validated — no overlaps detected
```



COMPLETE EXAMPLE

Putting It All Together

```
1  {
2    id: 'my-slide',
3    background: '#0a0a1a',
4    elements: [
5      el('<h1>Title</h1>', {
6        x: 960, y: 200,
7        w: 800, anchor: 'tc'
8      }),
9      hstack([card1, card2, card3], {
10       x: 120, y: 400,
11       gap: 'lg'
12     }),
13   ]
14 }
```



A complete slide is a plain object — no magic, just coordinates.

Start Building

Every slide in this presentation was built with the library it describes.

🎯

Deterministic

Same input, same output. Every time.

📐

Measurable

Know exact sizes before rendering.

✅

Validated

Catch errors before you see them.

github.com/cabird/slidekit